TypeScript

Session 2

Objectives

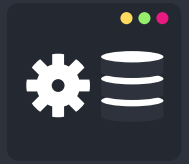
By the end of this session, You will be able to:

1. Understand why TypeScript exists and how it evolved.
2. Learn the difference between static vs dynamic typing.
3. Explore TypeScript's main features (types, inference, interfaces, generics).
4. Learn what transpilers do and how TSC works.
5. Configure TypeScript using tsconfig.json.
6. See how TypeScript fits into modern build workflows

Recap

- What happened last Session?
- What did we talk about?
- What was the benefit you got from last session?

Table of contents



01 Recap

02 Tour in our Resources

03 What is Backend?

04 Backend Engineer Mindset

05 Languages & Tools Overview

06 JavaScript Deep Dive

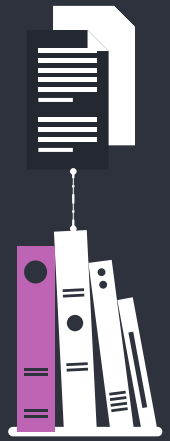


Table of Content

1_ Problem with JS

2_ How the Problem is Solved

3_ What is TypeScript

4_ Basics of TypeScript

- ★ Types
- ★ Union Types
- ★ Functions
- ★ Arrays
- ★ Generic Types
- ★ Interfaces
- ★ Intersection Types
- ★ classes
- ★ Type vs Interface vs Class vs Object
- ★ Utility Types

Table of Content

5_ code Structure Different Philosophies

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

</ JS development:

- JavaScript is awesome for building things, but it can get chaotic.
- Suddenly you're debugging for hours, and the bug?
- Something JS didn't warn you about until it exploded at runtime.

How was this problem fixed?

</ TypeScript was the answer

Microsoft introduced TypeScript as a solution:

- Code is safe and type annotated
- Bugs appear early in compile time
- Maintaining Large projects became easier.

What is TypeScript?

</ TypeScript, JavaScript's cousin

TypeScript is a superset of JavaScript, Where it includes static types, and can also be dynamically typed.

TypeScript is also not "Compiled" but rather Transpiled – converted into JavaScript code, but why?

</ Quick fact

- When TypeScript was created, no browser or runtime could execute TypeScript directly.
- So TypeScript had two choices:
  - Option A:
    - Create a new runtime or virtual machine that understands TypeScript → Not practical.
    - It would break compatibility with the entire JS ecosystem.
  - Option B:
    - Compile TypeScript into JavaScript, which browsers already understand.
    - TypeScript chose 100% compatibility with existing JavaScript engines, so it had to be transpiled.

</ Dynamic vs Static Typing

- Dynamic typing means type checking happens at runtime, not during compilation. (e.g. Python, JavaScript)

- While Static typing is that the type checking happens during writing code, so errors appear early on. (e.g. C++, Java)

Quick Competition

Why do we say compilation time even tho JS is interpreted?

First one to answer => 5 mohsens

Table of Content

~~1_ Problem with JS~~

~~2_ How the Problem is Solved~~

~~3_ What is TypeScript~~

4_ Basics of TypeScript

- ★ Types
- ★ Union Types
- ★ Functions
- ★ Arrays
- ★ Generic Types
- ★ Interfaces
- ★ Intersection Types
- ★ classes
- ★ Type vs Interface vs Class vs Object
- ★ Utility Types

TypeScript Basics

1- Types:

```
let Name: string = "Mohsen";
let Age: number = 28;
let isCool: Boolean = true;
```

You can also define your own type, for example:

```
type Committee = "Backend" | "Linux" | "Frontend";
```

extra- WD-40 of TypeScript:

The Any keyword is used when you don't expect a certain data type, like what JavaScript does.

Example:

```
let age: any;
age = "hello world";
age = 32;
```

2- Union Types:

```
let age: Number | String = "8";
```

As you can see we can here use the pipe operator " | " to declare that a variable may have 2 different data types.

3- Functions:

```
function functionName(Parameters: parameterType): ReturnType {
    Function body
}
```

Example:

```
function Greet(name: string): void {
   console.log(`Hello ${name}`)
}
```

3- Arrow function

Two Ways:

Option 1: const fun_name : function_type = (parms) => {Body}

```
const sum:(x:number,y:number)=>number = (x,y)=>{
return x+y
}
```

3- Arrow function

Option 1: const fun_name : function_type = (parms) => {Body}

Meaning

You are telling TypeScript:

"The variable sum MUST be a function that takes two numbers and returns a number."

Key idea

- You type the variable sum
- Before even writing the function body, TypeScript already knows the full function shape.

Why used?

- When you want to strictly control the exact function type
- Useful for callbacks, interfaces, reusable function types

3- Arrow function

Two Ways:

Option 2: const fun_name = (parms) : ret_type => {Body}

```
const fun = (x:number,y:number):number=>{return x+y}
```

3- Arrow function

Option 2: const fun_name = (parms) : ret_type => {Body}

Meaning

- You only add types inside the function (parameter + return).
- TypeScript automatically figures out:

  "fun is a function that takes a number and returns a string."

Key idea

- You type the function itself, not the variable
- The variable fun gets its type from the function

Why used?

- Cleaner
- Shorter
- Type inference does most of the work

```javascript
function CalculateArea [Length, Width] {
   return Length*Width;
}

let area = CalculateArea[2,3]; // Works fine
area = CalculateArea[2,"3"];   // output???
```

JavaScript

```typescript
function CalculateArea [Length: number, Width: number]: number {
   return Length*Width;
}

let area = CalculateArea[2,3];   // Works fine
area = CalculateArea[2,"3"];    // output??
```

TypeScript

4- Arrays

```
let arr =[1,2,3,"hi",true];

let ar:any[] =[1,2,3,"hi",true]; //same as the previous

let array:(string | number )[]  =[1,2,3,"hi"];

let a:(string )[]  =["yomna","hi"];

let nums:(number )[]  =[1,2,3];
```

4- Arrays

```
let tuple: [string, number, boolean] = ["Yomna", 25, true];
// Tuple: fixed number of elements with fixed ordered types
//unlike a union, which allows one type OR another in a single value
let readonlyArr: readonly number[] = [1, 2, 3];
// can't modify the array (push, pop,etc..)
let nums: Array<number> = [1, 2, 3];

let names: Array<string> = ["Yomna", "Ali"];

let mixed: Array<number|string> = [1, "hi"];
```

5- Generic Types:

```
function identity<T>(value: T): T {
   return value;
}

let num = identity<number>(42);     // T is number
let str = identity<string>("hello"); // T is string
let auto = identity(true);          // T inferred as boolean
```

This is like generalizing what a function can do to multiple data types.

Table of Content

1_ ~~Problem with JS~~

2_ ~~How the Problem is Solved~~

3_ ~~What is TypeScript~~

4_ Basics of TypeScript

- ~~Types~~
- ~~Union Types~~
- ~~Functions~~
- ~~Arrays~~
- ~~Generic Types~~
- Interfaces
- Intersection Types
- classes
- Type vs Interface vs Class vs Object
- Utility Types

Remember Objects?

6- Interfaces:

Think of them as Structs in C++

```
interface member {
    name: String;
    age: Number;
    committee: String;
}
```

When this code compiled into JS , it will be:

```
var mem = {
  name: "Mohsen",
  age: 28,
  committee: "BackEnd"
};
console.log(mem);
```

There is no interface in JavaScript, and TypeScript annotations (: Member) are removed after compilation

Discriminated Unions:

```
interface Circle {
 type: "circle";
 radius: number;}

interface Square {
 type: "square";
 side: number;}

type Shape = Circle | Square;
function area(s: Shape) {
 if (s.type === "circle") return Math.PI * s.radius ** 2;
 return s.side * s.side; }
```

7- Intersection Types: “Type Aliases”

```
type Id = { id: number };
type Name = { name: string };
type User = Id & Name;
let person:User={id:10,name:"yomna"}
```

Here use the ampersand operator “ & ” to declare that a variable will have the attributes of all types declared.

8- classes

Class = a blueprint for creating **objects.**

Can contain:

1. **Properties** (object variables).
2. **Constructor** (function executed when an object is created).
3. **Methods** (functions inside the object).

9- TypeScript: Type vs Interface vs Class vs Object

Type	Interface	Class	Object
Compile-Time Only	Compile-Time Only	Run time	Run time
Disappears after compilation as it is unique for TS	Disappears after compilation as it is unique for TS	Exists at runtime, in the JS code	Exists at runtime, in the JS code
A way to define shapes and aliases for type checking	A contract that defines object structure	A blueprint for creating objects with behavior	Actual data stored in memory

10- Utility Types

1. Partial Type
2. Required Type
3. Readonly Type
4. Pick Type
5. Omit Type

1- partial <type>

makes **all properties of a type optional.**

```
interface User {
 id: number;
 name: string;
 email: string; }

const user2: User = {
 id: 2,
 name: "Ali"
}; // Error: Property 'email' is missing
```

```
const partial_User: Partial<User> = {id: 3};
// Works, name and email are optional
```

```
## We can use "?" in interfaces => it means optional
attribute

interface User {
 id: number;
 name?: string; // Works, name is optional
 email: string;
}
```

2-Required <Type>

makes **all properties of a type** Required.

```
interface User{
 id:number,
 name:string,
 email?:string
}
const user1:Required<User> ={
 id:2,
 name:"yomna",
 email:" yomna@gmail.com" }
```

3-Readonly <Type>

makes **all properties of a type** Required.

```
interface User{
 id:number,
 name:string,
 email?:string }

const user2:Readonly<User> ={
 id:2,
 name:"yomna",
 email: "yomna@gmail.com" }

user2.id=3 //Error ,Cannot assign to 'id' because it is a read-only property
```

4-Pick <Type>

Select some **properties**.

```
interface User{
 id:number,
 name:string,
 email?:string
}
const user3:Pick<User ,"name"|"email"> ={
 Id:2,   //id' does not exist in type 'Pick<User, "name" | "email">'
 name:"yomna"
}
```

5-Omit <Type>

drops **properties**

```
interface User{
 id:number,
 name:string,
 email?:string }

const user4:Omit<User ,"name"> ={
 id:2,
 email:"yomna@gmail.com",
 name:"yomna" // error ,'name' does not exist in type 'Omit<User, "name"> }
```

Table of Content

1_ ~~Problem with JS~~

2_ ~~How the Problem is Solved~~

3_ ~~What is TypeScript~~

4_ ~~Basics of TypeScript~~

- ★ Types
- ★ ~~Union Types~~
- ★ Functions
- ★ Arrays
- ★ ~~Generic Types~~
- ★ ~~Interfaces~~
- ★ ~~Intersection Types~~
- ★ classes
- ★ ~~Type vs Interface vs Class vs Object~~
- ★ ~~Utility Types~~

Table of Content

5_ code Structure Different Philosophies

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

</ Top-down vs Bottom-up flows

Top-Down

```
interface User {
    id: number;
    name: string;
    address: Address;
    roles: Role[];
}
interface Address {
    city: string;
    street: string;
}
type Role = "admin" | "member" | "guest";
```

Bottom-up

```
interface Id {
    id: number;
}
interface Name {
    name: string;
}
type Role = "admin" | "member" | "guest";

interface User extends Id, Named {
    roles: Role[];}
```

Bottom up	Top down
Start small then build up	Start big then break down
Build all pieces then assemble	Write what you want first then define how later

</ Shape Compatibility

If we define multiple interfaces that have common attributes we can assign an object of the more general interface to an object of the narrower scope interface.

Shape Compatibility example

```
interface Person {
  name: string;
  age: number; }

interface Employee {
  name: string;
  age: number;
  employeeId: number; }

const p: Person = {
  name: "Bianka",
  age: 5 };
const e: Employee = p; // Error
```

```
const emp: Employee = {
  name: "Bianka",
  age: 5,
  employeeId: 101
};

const person: Person = emp; // works
```

How to make it work in 4 cases!
++ mohsens!

Table of Content

~~5_ code Structure Different Philosophies~~

6_ How TS is Transpiled

7_ How to set it up

8_ What Files Matters to us

Hands On

Break